Lab Assignment 1
Object Oriented Programming (IT620)

Name: Aman Rohera

Student ID:202512049

Q1)

```python
def sum_of_all(arr):
    total = 0
    for num in arr:
        total += num
    return total

def max_element(arr):
    if not arr:
        return None
    maximum = arr[0]
    for num in arr:
        if num > maximum:
            maximum = num
    return maximum

def count_even(arr):
    count = 0
    for num in arr:
        if num % 2 == 0:
            count += 1
    return count


n = int(input("Enter the number of elements: "))
arr=[]
for i in range(n):
    element = int(input(f"Enter element {i+1}: "))
    arr.append(element)
result = sum_of_all(arr)
print(f"Sum of all elements: {result}")
result = max_element(arr)
print(f"Maximum element: {result}")
result = count_even(arr)
print(f"Count of even elements: {result}")
```

Basically, for the summation of the numbers, I iterate the loop and create a variable that stores the sum, and add the number to that variable
For maximum, I check every element against another element.
For even numbers, I perform the modulo operation on every number.

```
D:\DAU\Sem 2\OOP\Practical\Lab 1>python Q1.py
Enter the number of elements: 8
Enter element 1: 1
Enter element 2: 5
Enter element 3: 6
Enter element 4: 4
Enter element 5: 12
Enter element 6: 25
Enter element 7: 28
Enter element 8: 30
Sum of all elements: 111
Maximum element: 30
Count of even elements: 5
```

Q2)

```python
def reverse_stack(stack):
    new_stack=[]
    if not stack:
        return []
    while(stack):
        top_element = stack.pop()
        new_stack.append(top_element)
    return new_stack


n = int(input("Enter the number of elements in the stack: "))
stack = []
for i in range(n):
    element = int(input(f"Enter element {i+1}: "))
    stack.append(element)
print("Reversed stack elements:")
reversed_stack = reverse_stack(stack)
print(reversed_stack)
```

To reverse the stack, I create an arbitrary stack, pop elements from the old stack until it's empty, and push each popped element into the new stack.

```
D:\DAU\Sem 2\OOP\Practical\Lab 1>python Q2.py
Enter the number of elements in the stack: 5
Enter element 1: 2
Enter element 2: 5
Enter element 3: 6
Enter element 4: 9
Enter element 5: 4
Reversed stack elements:
[4, 9, 6, 5, 2]
```

Q3)

```python
def permutations(lst):
    if len(lst) <= 1:
        return [lst]

    result = []
    for i in range(len(lst)):
        rest = lst[:i] + lst[i+1:]
        for p in permutations(rest):
            result.append([lst[i]] + p)
    return result


n=int(input("Enter the number of elements: "))
lst=[]
for i in range(n):
    element=int(input(f"Enter element {i+1}: "))
    lst.append(element)
print("Input:", lst)
print("Output:", permutations(lst))
```

In this permutation, if we have 3 element,s so 1 element is fixed and we permute the rest 2 through a loop

```
D:\DAU\Sem 2\OOP\Practical\Lab 1>python Q3.py
Enter the number of elements: 3
Enter element 1: 1
Enter element 2: 2
Enter element 3: 3
Input: [1, 2, 3]
Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

D:\DAU\Sem 2\OOP\Practical\Lab 1>python Q3.py
Enter the number of elements: 4
Enter element 1: 2
Enter element 2: 5
Enter element 3: 6
Enter element 4: 9
Input: [2, 5, 6, 9]
Output: [[2, 5, 6, 9], [2, 5, 9, 6], [2, 6, 5, 9], [2, 6, 9, 5], [2, 9, 5, 6], [2, 9, 6, 5], [5, 2, 6, 9], [5, 2, 9, 6],
 [5, 6, 2, 9], [5, 6, 9, 2], [5, 9, 2, 6], [5, 9, 6, 2], [6, 2, 5, 9], [6, 2, 9, 5], [6, 5, 2, 9], [6, 5, 9, 2], [6, 9,
2, 5], [6, 9, 5, 2], [9, 2, 5, 6], [9, 2, 6, 5], [9, 5, 2, 6], [9, 5, 6, 2], [9, 6, 2, 5], [9, 6, 5, 2]]
```

Q4)

```python
def mountain_array(arr):
    n = len(arr)
    if(n < 3):
        return "Not a Mountain Array"
    peak_index = 0
    for i in range(1, n - 1):
        if arr[i] > arr[i - 1] and arr[i] > arr[i + 1]:
            peak_index = i
            break
    if(peak_index == 0):
        return "Not a Mountain Array"
    for i in range(peak_index):
        if arr[i] >= arr[i + 1]:
            return "Not a Mountain Array"
    for i in range(peak_index, n - 1):
        if arr[i] <= arr[i + 1]:
            return "Not a Mountain Array"
    return f"Valid Mountain, Peak = {arr[peak_index]}"


n=int(input("Enter the number of elements: "))
arr=[]
for i in range(n):
    element=int(input(f"Enter element {i+1}: "))
    arr.append(element)
print(mountain_array(arr))
```

In this, we first find the peak element. After that, we iterate over the range [0, peak] and check whether all elements are in increasing order, and over the range [peak, n-1] whether all elements are in decreasing order.

```
Enter the number of elements: 8
Enter element 1: 2
Enter element 2: 3
Enter element 3: 4
Enter element 4: 12
Enter element 5: 8
Enter element 6: 7
Enter element 7: 6
Enter element 8: 5
Valid Mountain, Peak = 12

D:\DAU\Sem 2\OOP\Practical\Lab 1>python Q4.py
Enter the number of elements: 5
Enter element 1: 2
Enter element 2: 12
Enter element 3: 3
Enter element 4: 4
Enter element 5: 5
Not a Mountain Array

D:\DAU\Sem 2\OOP\Practical\Lab 1>python Q4.py
Enter the number of elements: 5
Enter element 1: 1
Enter element 2: 2
Enter element 3: 12
Enter element 4: 12
Enter element 5: 3
Not a Mountain Array
```

Q5)

```python
def char_frequency(s):
    freq = {}
    for ch in s:
        if ch in freq:
            freq[ch] += 1
        else:
            freq[ch] = 1
    result = {}
    for ch in 'abcdefghijklmnopqrstuvwxyz':
        if ch in freq:
            result[ch] = freq[ch]
    return result


string = "adcbbdaacd"
print("Input:", string)
print("Output:", char_frequency(string))
```

Here simple dictionary is applied; we iterate over all elements and increment the counter. And then I take a string and check whether it's present in the unsorted dictionary. If so, we insert it into the new dictionary.

```
Input: adcbbdaacd
Output: {'a': 3, 'b': 2, 'c': 2, 'd': 3}
```

Q6)

```python
def find_n_largest_smallest(arr, n):
    largest = []
    smallest = []
    for k in range(n):
        min_val = arr[0]
        for i in range(len(arr)):
            if arr[i] < min_val and arr[i] not in smallest:
                min_val = arr[i]
        smallest.append(min_val)
    for k in range(n):
        max_val = arr[0]
        for i in range(len(arr)):
            if arr[i] > max_val and arr[i] not in largest:
                max_val = arr[i]
        largest.append(max_val)

    print("Largest =", largest)
    print("Smallest =", smallest)


# Taking input from user
n = int(input("Enter N: "))
arr = []
size = int(input("Enter the number of elements in the array: "))
for i in range(size):
    element = int(input(f"Enter element {i+1}: "))
    arr.append(element)

find_n_largest_smallest(arr, n)
```

We take a n and iterate the main loop and find the largest and smallest elements.

```
D:\DAU\Sem 2\OOP\Practical\Lab 1>python Q6.py
Enter N: 2
Enter the number of elements in the array: 5
Enter element 1: 1
Enter element 2: 2
Enter element 3: 3
Enter element 4: 4
Enter element 5: 5
Largest = [5, 4]
Smallest = [1, 1]
```